Contents

1. Range Queries

1.1. Static Range Sum Queries

Question - Static Range Sum Queries Backup Link

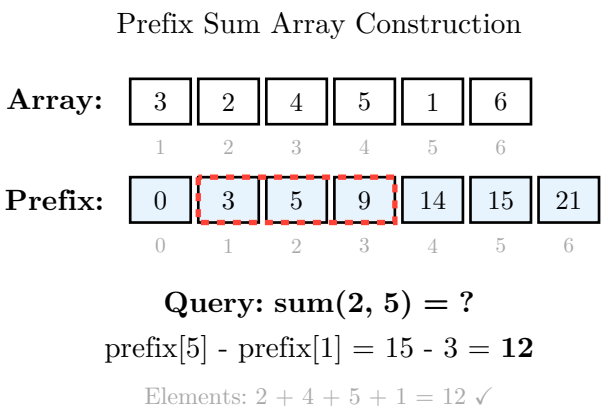
Explanation :

Given an array of `n` integers and `q` queries, each query asks for the sum of elements in a range `[a, b]`. A naive approach would iterate through each range, giving O(n) per query and O(n·q) total — too slow for large inputs.

The Key Insight - Prefix Sums: Instead of summing ranges repeatedly, we precompute a prefix sum array where `prefix[i]` = sum of elements from index 1 to i. Then any range sum can be computed in O(1):

```
sum(a, b) = prefix[b] - prefix[a-1]
```

Why This Works: `prefix[b]` contains the sum of elements 1 to b. By subtracting `prefix[a-1]` (sum of elements 1 to a-1), we're left with exactly the sum from a to b.



Building the Prefix Array:

```
Array:    [3, 2, 4, 5, 1, 6]  (1-indexed)
prefix[0] = 0                  (base case)
prefix[1] = prefix[0] + arr[1] = 0 + 3 = 3
prefix[2] = prefix[1] + arr[2] = 3 + 2 = 5
prefix[3] = prefix[2] + arr[3] = 5 + 4 = 9
prefix[4] = prefix[3] + arr[4] = 9 + 5 = 14
prefix[5] = prefix[4] + arr[5] = 14 + 1 = 15
prefix[6] = prefix[5] + arr[6] = 15 + 6 = 21
```

Query Examples:

```
sum(1, 3) = prefix[3] - prefix[0] = 9 - 0 = 9   (3+2+4)
```

```
sum(2, 5) = prefix[5] - prefix[1] = 15 - 3 = 12 (2+4+5+1)
sum(4, 6) = prefix[6] - prefix[3] = 21 - 9 = 12 (5+1+6)
sum(3, 3) = prefix[3] - prefix[2] = 9 - 5 = 4   (just element 3)
```

Time Complexity:

- Preprocessing: O(n) to build prefix array
- Each query: O(1)
- Total: O(n + q)

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    cin >> n >> q;

    vector<ll> arr(n + 1);
    vector<ll> prefix(n + 1, 0);

    // Read array and build prefix sums
    for (int i = 1; i <= n; i++) {
        cin >> arr[i];
        prefix[i] = prefix[i - 1] + arr[i];
    }

    // Answer queries
    while (q--) {
        int a, b;
        cin >> a >> b;
        cout << prefix[b] - prefix[a - 1] << "\n";
    }

    return 0;
}
```

1.2. Static Range Minimum Queries

Question - Static Range Minimum Queries Backup Link

Explanation :

Given an array of `n` integers and `q` queries, each query asks for the minimum element in a range `[a, b]`. Unlike range sums, we cannot simply use prefix arrays because `min(a,b) - min(a,c) ≠ min(c+1,b)`. We need a different approach: **Sparse Table**.

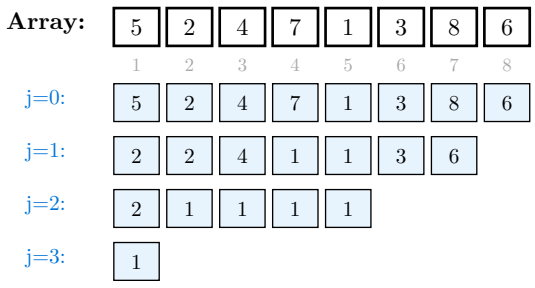
Why Sparse Table? Sparse Table is perfect for **static** range queries on **idempotent** operations (where `f(x, x) = x`). Minimum is idempotent: `min(x, x) = x`. This allows overlapping ranges without double-counting, enabling O(1) queries.

The Core Idea: Precompute answers for all ranges of length 2^j. Any range can be covered by at most 2 overlapping power-of-2 ranges. For minimum, overlapping is fine!

Building the Sparse Table:

- `sparse[i][j]` = minimum in range starting at index i with length 2^j
- Base case: `sparse[i][0] = arr[i]` (ranges of length 1)
- Recurrence: `sparse[i][j] = min(sparse[i][j-1], sparse[i + 2^(j-1)][j-1])`

Sparse Table Construction



sparse[i][j] = min of 2^j elements starting at i

sparse[1][2] = min(5,2,4,7) = 2

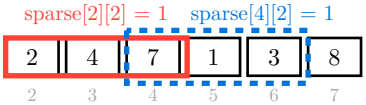
sparse[5][1] = min(1,3) = 1

Answering Queries in O(1): For range `[L, R]`, find the largest `k` such that $2^k \leq R - L + 1$. Then:

```
1  answer = min(sparse[L][k], sparse[R - 2^k + 1][k])
```

These two ranges of length 2^k together cover `[L, R]` completely (with possible overlap, which is fine for minimum).

Query: min(2, 7) on array [5, 2, 4, 7, 1, 3, 8, 6]



k = floor(log2(7-2+1)) = floor(log2(6)) = 2

Answer = min(sparse[2][2], sparse[4][2]) = min(1, 1) = **1**

Time Complexity:

- Preprocessing: O(n log n) to build sparse table
- Each query: O(1)
- Space: O(n log n)

Code :

```C++
#include <bits/stdc++.h>
using namespace std;

const int MAXLOG = 20;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    cin >> n >> q;

    vector<int> arr(n + 1);
    for (int i = 1; i <= n; i++) {
        cin >> arr[i];
    }

    // Build sparse table
    // sparse[i][j] = minimum in range [i, i + 2^j - 1]
    vector<vector<int>> sparse(n + 1, vector<int>(MAXLOG));

    // Base case: ranges of length 1
    for (int i = 1; i <= n; i++) {
        sparse[i][0] = arr[i];
    }

    // Fill for larger powers of 2
    for (int j = 1; j < MAXLOG; j++) {
        for (int i = 1; i + (1 << j) - 1 <= n; i++) {
            sparse[i][j] = min(sparse[i][j - 1],
                               sparse[i + (1 << (j - 1))][j - 1]);
        }
    }

    // Precompute log values for O(1) query
    vector<int> lg(n + 1);
    lg[1] = 0;
    for (int i = 2; i <= n; i++) {
        lg[i] = lg[i / 2] + 1;
```

```
    }

    // Answer queries
    while (q--) {
        int a, b;
        cin >> a >> b;

        int len = b - a + 1;
        int k = lg[len];

        // Two overlapping ranges of length 2^k cover [a, b]
        int ans = min(sparse[a][k], sparse[b - (1 << k) + 1][k]);
        cout << ans << "\n";
    }

    return 0;
}
```

1.3. Dynamic Range Sum Queries

Question - Dynamic Range Sum Queries Backup Link

Explanation :

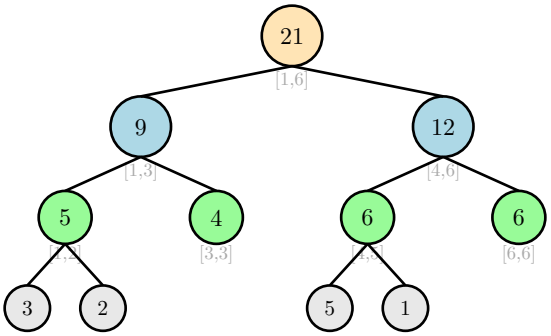
Now we have **updates**: change the value at position `k` to `u`, then answer sum queries on ranges. Prefix sums won't work anymore — updating one element would require rebuilding the entire prefix array in O(n). We need a **Segment Tree**.

What is a Segment Tree? A segment tree is a binary tree where each node stores information about a range of the array. The root covers the entire array, and each node's children split its range in half. Leaves represent individual elements.

Key Properties:

- Tree has O(n) nodes (at most 4n for safety)
- Height is O(log n)
- Each update affects O(log n) nodes (ancestors of the leaf)
- Each query visits O(log n) nodes

Segment Tree for array [3, 2, 4, 5, 1, 6]



Each node stores sum of its range

Parent = left child + right child

Query Operation - sum(2, 5): We recursively find nodes whose ranges are completely inside `[2, 5]` and sum them up.

Query sum(2, 5): Find nodes covering [2,5]

Node [1,3]?
Partially overlaps
→ Go to children

Node [2,2] ✓
Fully inside [2,5]
→ Return 2

Node [4,5] ✓
Fully inside [2,5]
→ Return 6

Node [3,3] ✓
Fully inside [2,5]
→ Return 4

Answer = 2 + 4 + 6 = 12

Update Operation - set position 3 to 10: Update the leaf, then propagate changes up to the root by recalculating parent sums.

```
1 Before: arr[3] = 4
```

```
After:  arr[3] = 10

Update path (bottom-up):
  Leaf [3,3]: 4 → 10
  Parent [1,3]: 9 → 9 - 4 + 10 = 15
  Root [1,6]: 21 → 21 - 4 + 10 = 27
```

Time Complexity:

- Build: O(n)
- Update: O(log n)
- Query: O(log n)

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

int n, q;
vector<ll> arr, tree;

// Build segment tree
void build(int node, int start, int end) {
    if (start == end) {
        tree[node] = arr[start];
    } else {
        int mid = (start + end) / 2;
        build(2 * node, start, mid);
        build(2 * node + 1, mid + 1, end);
        tree[node] = tree[2 * node] + tree[2 * node + 1];
    }
}

// Point update: set arr[idx] = val
void update(int node, int start, int end, int idx, ll val) {
    if (start == end) {
        arr[idx] = val;
        tree[node] = val;
    } else {
        int mid = (start + end) / 2;
        if (idx <= mid) {
            update(2 * node, start, mid, idx, val);
        } else {
```

```
            update(2 * node + 1, mid + 1, end, idx, val);
        }
        tree[node] = tree[2 * node] + tree[2 * node + 1];
    }
}

// Range sum query [l, r]
ll query(int node, int start, int end, int l, int r) {
    if (r < start || end < l) {
        return 0;  // Out of range
    }
    if (l <= start && end <= r) {
        return tree[node];  // Fully inside
    }
    int mid = (start + end) / 2;
    ll left_sum = query(2 * node, start, mid, l, r);
    ll right_sum = query(2 * node + 1, mid + 1, end, l, r);
    return left_sum + right_sum;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n >> q;

    arr.resize(n + 1);
    tree.resize(4 * n);

    for (int i = 1; i <= n; i++) {
        cin >> arr[i];
    }

    build(1, 1, n);

    while (q--) {
        int type;
        cin >> type;

        if (type == 1) {
            int k;
            ll u;
            cin >> k >> u;
            update(1, 1, n, k, u);
        } else {
```

```
76              int a, b;
77              cin >> a >> b;
78              cout << query(1, 1, n, a, b) << "\n";
79          }
80      }
81
82      return 0;
83  }
```

1.4. Dynamic Range Minimum Queries

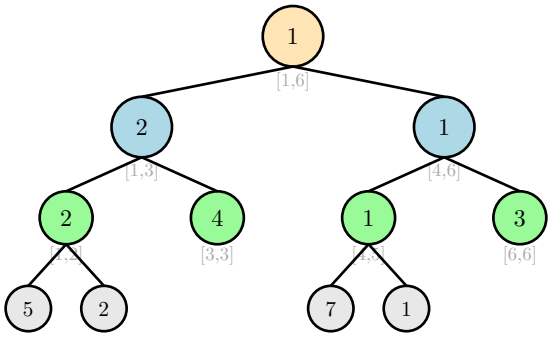
Question - Dynamic Range Minimum Queries Backup Link

Explanation :

This is nearly identical to Dynamic Range Sum Queries, but instead of storing sums in segment tree nodes, we store minimums. The key difference is in how we combine children and what we return for out-of-range queries.

Changes from Sum to Minimum:
1. **Combine operation**: `tree[node] = min(left_child, right_child)` instead of sum
2. **Identity element**: Return ∞ (a large value) for out-of-range queries instead of 0
3. **Node meaning**: Each node stores the minimum value in its range

Segment Tree for Minimum: array [5, 2, 4, 7, 1, 3]



Each node stores min of its range
Parent = min(left child, right child)

Why Return Infinity for Out-of-Range? When a range doesn't overlap with our query, it shouldn't affect the answer. For minimum, we return ∞ because `min(x, ∞) = x` — it doesn't change the result. This is the **identity element** for the min operation.

Query Example - min(2, 5):

```
Query [2,5]:
  Node [1,6]: partial overlap → recurse
    Node [1,3]: partial overlap → recurse
      Node [1,2]: partial → recurse
        Node [1,1]: outside → return ∞
        Node [2,2]: inside → return 2
      Node [3,3]: inside → return 4
    Node [4,6]: partial overlap → recurse
      Node [4,5]: inside → return 1
      Node [6,6]: outside → return ∞

Result: min(∞, 2, 4, 1, ∞) = 1
```

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll INF = 1e18;

int n, q;
vector<ll> arr, tree;

// Build segment tree for minimum
void build(int node, int start, int end) {
    if (start == end) {
        tree[node] = arr[start];
    } else {
        int mid = (start + end) / 2;
        build(2 * node, start, mid);
        build(2 * node + 1, mid + 1, end);
        tree[node] = min(tree[2 * node], tree[2 * node + 1]);
    }
}

// Point update: set arr[idx] = val
void update(int node, int start, int end, int idx, ll val) {
    if (start == end) {
        arr[idx] = val;
        tree[node] = val;
    } else {
        int mid = (start + end) / 2;
        if (idx <= mid) {
            update(2 * node, start, mid, idx, val);
        } else {
            update(2 * node + 1, mid + 1, end, idx, val);
        }
        tree[node] = min(tree[2 * node], tree[2 * node + 1]);
    }
}

// Range minimum query [l, r]
ll query(int node, int start, int end, int l, int r) {
    if (r < start || end < l) {
        return INF;  // Out of range - return identity
    }
    if (l <= start && end <= r) {
        return tree[node];  // Fully inside
    }
```

```
    int mid = (start + end) / 2;
    ll left_min = query(2 * node, start, mid, l, r);
    ll right_min = query(2 * node + 1, mid + 1, end, l, r);
    return min(left_min, right_min);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n >> q;

    arr.resize(n + 1);
    tree.resize(4 * n);

    for (int i = 1; i <= n; i++) {
        cin >> arr[i];
    }

    build(1, 1, n);

    while (q--) {
        int type;
        cin >> type;

        if (type == 1) {
            int k;
            ll u;
            cin >> k >> u;
            update(1, 1, n, k, u);
        } else {
            int a, b;
            cin >> a >> b;
            cout << query(1, 1, n, a, b) << "\n";
        }
    }

    return 0;
}
```

1.5. Range Xor Queries

Question - Range Xor Queries Backup Link

Explanation :

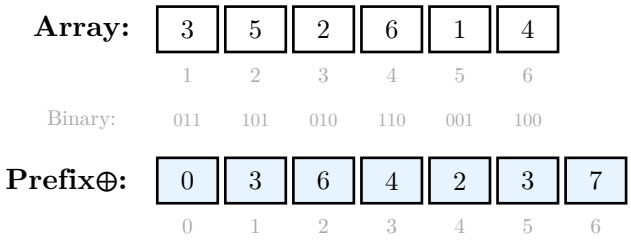
Given an array and queries asking for the XOR of elements in range `[a, b]`, this problem is static (no updates). We can use the same prefix technique as range sums, but with XOR instead of addition.

The Magic Property of XOR: XOR has a special property: `x ⊕ x = 0` (any number XORed with itself is 0). This means:

```
prefix[b] ⊕ prefix[a-1] = (arr[1] ⊕ ... ⊕ arr[b]) ⊕ (arr[1] ⊕ ... ⊕ arr[a-1])
                        = arr[a] ⊕ arr[a+1] ⊕ ... ⊕ arr[b]
```

The elements from 1 to a-1 appear in both and cancel out!

Prefix XOR Array



Query: XOR(2, 5) = ?

$\text{prefix}[5] \oplus \text{prefix}[1] = 3 \oplus 3 = \mathbf{0}$

Verify: $5 \oplus 2 \oplus 6 \oplus 1 = 101 \oplus 010 \oplus 110 \oplus 001 = 000$ ✓

Building Prefix XOR:

```
Array:      [3, 5, 2, 6, 1, 4]
prefix[0] = 0                          (base case)
prefix[1] = prefix[0] ⊕ arr[1] = 0 ⊕ 3 = 3
prefix[2] = prefix[1] ⊕ arr[2] = 3 ⊕ 5 = 6
prefix[3] = prefix[2] ⊕ arr[3] = 6 ⊕ 2 = 4
prefix[4] = prefix[3] ⊕ arr[4] = 4 ⊕ 6 = 2
prefix[5] = prefix[4] ⊕ arr[5] = 2 ⊕ 1 = 3
prefix[6] = prefix[5] ⊕ arr[6] = 3 ⊕ 4 = 7
```

Query Examples:

```
XOR(1, 3) = prefix[3] ⊕ prefix[0] = 4 ⊕ 0 = 4   (3⊕5⊕2 = 4)
XOR(2, 4) = prefix[4] ⊕ prefix[1] = 2 ⊕ 3 = 1   (5⊕2⊕6 = 1)
XOR(3, 6) = prefix[6] ⊕ prefix[2] = 7 ⊕ 6 = 1   (2⊕6⊕1⊕4 = 1)
```

Time Complexity:

- Preprocessing: O(n)
- Each query: O(1)

Code :

```C++
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    cin >> n >> q;

    vector<int> arr(n + 1);
    vector<int> prefix(n + 1, 0);

    // Build prefix XOR array
    for (int i = 1; i <= n; i++) {
        cin >> arr[i];
        prefix[i] = prefix[i - 1] ^ arr[i];
    }

    // Answer queries
    while (q--) {
        int a, b;
        cin >> a >> b;
        cout << (prefix[b] ^ prefix[a - 1]) << "\n";
    }

    return 0;
}
```

1.6. Range Update Queries

Question - Range Update Queries Backup Link

Explanation :

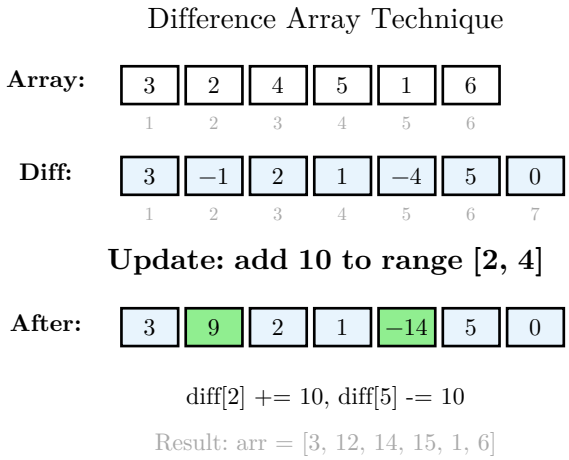
This problem flips our previous pattern: now we have **range updates** and **point queries**. We need to add a value `u` to all elements in range `[a, b]`, then query individual element values.

The Naive Approach Problem: Updating each element in a range takes O(n) per update. With many updates, this is too slow.

The Clever Solution - Difference Array: Instead of storing actual values, we store the **difference** between consecutive elements. Then a range update `[a, b] += u` becomes just two point updates:

- `diff[a] += u` (start adding u from position a)
- `diff[b+1] -= u` (stop adding u after position b)

To get the actual value at position k, we compute the prefix sum of the difference array up to k.



Using a BIT/Fenwick Tree: To efficiently compute prefix sums for point queries, we use a Binary Indexed Tree (BIT). This gives us:

- Range update in O(log n): update two positions in BIT
- Point query in O(log n): query prefix sum at position k

How BIT Works: A BIT stores partial sums in a clever way using binary representations. Position i in the BIT is responsible for a range of elements determined by the lowest set bit of i.

```
1 Update [a, b] += u:
2   BIT.add(a, +u)    // Start adding u from position a
3   BIT.add(b+1, -u)  // Stop adding u after position b
4
5 Query position k:
6   return BIT.prefixSum(k)  // Sum of all updates affecting position k
```

Example Trace:

```
Initial array: [3, 2, 4, 5, 1, 6]
We maintain a BIT for the difference array

Update 1: add 5 to [2, 4]
  BIT.add(2, +5)
  BIT.add(5, -5)

Update 2: add 3 to [1, 3]
  BIT.add(1, +3)
  BIT.add(4, -3)

Query position 3:
  Original value: 4
  + prefixSum(3) = 3 + 5 = 8
  Answer: 4 + 8 = 12
```

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

int n, q;
vector<ll> arr;
vector<ll> bit;  // BIT for difference array

// Add val to position i
void update(int i, ll val) {
    for (; i <= n; i += i & (-i)) {
        bit[i] += val;
    }
}

// Get prefix sum [1, i]
ll query(int i) {
    ll sum = 0;
    for (; i > 0; i -= i & (-i)) {
        sum += bit[i];
    }
    return sum;
}
```

```cpp

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n >> q;

    arr.resize(n + 1);
    bit.resize(n + 2, 0);  // Extra space for b+1 updates

    for (int i = 1; i <= n; i++) {
        cin >> arr[i];
    }

    while (q--) {
        int type;
        cin >> type;

        if (type == 1) {
            // Range update: add u to [a, b]
            int a, b;
            ll u;
            cin >> a >> b >> u;
            update(a, u);
            update(b + 1, -u);
        } else {
            // Point query: get value at position k
            int k;
            cin >> k;
            cout << arr[k] + query(k) << "\n";
        }
    }

    return 0;
}
```

1.7. Forest Queries

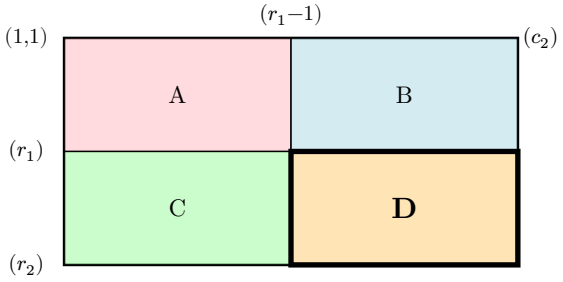
Question - Forest Queries Backup Link

Explanation :

Given an `n × n` grid where each cell is either a tree (*) or empty (.), answer queries asking how many trees are in a rectangular subgrid from (r_1, c_1) to (r_2, c_2). This is a 2D extension of prefix sums.

2D Prefix Sum Concept: Define `prefix[i][j]` = count of trees in the rectangle from $(1, 1)$ to (i, j). Using inclusion-exclusion, we can compute any rectangle sum in O(1).

2D Prefix Sum: Inclusion-Exclusion



Query (r1, c1) to (r2, c2):

sum(D) = prefix[r2][c2] - prefix[r1-1][c2]

- prefix[r2][c1-1] + prefix[r1-1][c1-1]

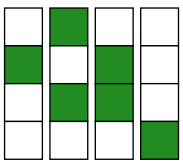
We subtract B and C, but A was subtracted twice, so add it back

Building the 2D Prefix Sum:

```
prefix[i][j] = grid[i][j]
             + prefix[i-1][j]
             + prefix[i][j-1]
             - prefix[i-1][j-1]
```

Example: 4×4 forest grid

Grid:



Prefix:

0	1	1	1
1	2	3	3
1	3	5	5
1	3	5	6

Query: trees in (2,2) to (4,4)?

2. prefix[4][4] - prefix[1][4] - prefix[4][1] + prefix[1][1]

3. 6 - 1 - 1 + 0 = 4 trees

Time Complexity:

- Preprocessing: $O(n^2)$ to build 2D prefix sum
- Each query: O(1)

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    cin >> n >> q;

    vector<string> grid(n + 1);
    vector<vector<int>> prefix(n + 1, vector<int>(n + 1, 0));

    // Read grid (1-indexed)
    for (int i = 1; i <= n; i++) {
        cin >> grid[i];
        grid[i] = " " + grid[i];  // Make 1-indexed
    }

    // Build 2D prefix sum
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            int tree = (grid[i][j] == '*') ? 1 : 0;
            prefix[i][j] = tree + prefix[i-1][j] + prefix[i][j-1] -
            prefix[i-1][j-1];
        }
    }

    // Answer queries
    while (q--) {
        int r1, c1, r2, c2;
        cin >> r1 >> c1 >> r2 >> c2;

        int ans = prefix[r2][c2]
                - prefix[r1-1][c2]
                - prefix[r2][c1-1]
                + prefix[r1-1][c1-1];

        cout << ans << "\n";
    }

    return 0;
}
```

3.1. Hotel Queries

Question - Hotel Queries Backup Link

Explanation :

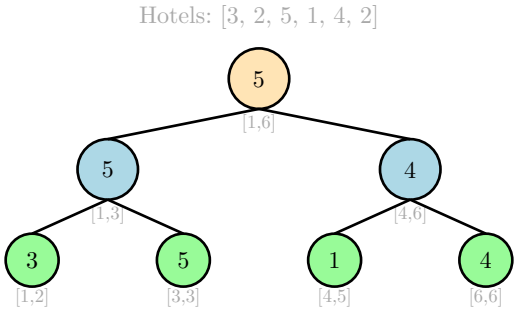
There are `n` hotels with given room capacities. For each of `m` groups, we need to assign them to the **first** hotel (leftmost) that has enough free rooms, then reduce that hotel's capacity.

The Naive Approach: For each group, scan hotels left to right until finding one with enough rooms. This is O(n) per query, giving O(n·m) total — too slow.

Segment Tree with Maximum: We use a segment tree where each node stores the **maximum** capacity in its range. This lets us efficiently find the leftmost hotel with sufficient capacity.

Key Insight: To find the leftmost hotel with capacity $\geq$ r:
1. If max of entire range $<$ r, no valid hotel exists $\rightarrow$ return 0
2. Otherwise, recursively search the left subtree first
3. If left subtree has a valid hotel, return it
4. Otherwise, search the right subtree

Segment Tree storing Maximum Capacities



Query Example - Group needs 4 rooms:

```
Search for leftmost hotel with capacity >= 4:

At root [1,6]: max=5 >= 4, so valid hotel exists
  → Check left child [1,3]: max=5 >= 4
    → Check left [1,2]: max=3 < 4, no valid hotel here
    → Check right [3,3]: max=5 >= 4, this is a leaf!
      → Return hotel 3

Hotel 3 had 5 rooms, group takes 4
Update hotel 3: capacity 5 → 1
Propagate max updates up the tree
```

After Assigning Group:

```
Hotel 3: 5 → 1
Node [3,3]: 5 → 1
```

```
3  Node [1,3]: max(3, 1) = 3
4  Node [1,6]: max(3, 4) = 4
```

Time Complexity:
- Each query: O(log n) to find hotel + O(log n) to update
- Total: O(m log n)

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

int n, m;
vector<int> hotel, tree;

void build(int node, int start, int end) {
    if (start == end) {
        tree[node] = hotel[start];
    } else {
        int mid = (start + end) / 2;
        build(2 * node, start, mid);
        build(2 * node + 1, mid + 1, end);
        tree[node] = max(tree[2 * node], tree[2 * node + 1]);
    }
}

// Find leftmost hotel with capacity >= rooms, and book it
int query(int node, int start, int end, int rooms) {
    if (tree[node] < rooms) {
        return 0;  // No valid hotel in this range
    }
    if (start == end) {
        // Found the hotel, book the rooms
        tree[node] -= rooms;
        return start;
    }

    int mid = (start + end) / 2;
    int result;

    // Try left subtree first (for leftmost)
    if (tree[2 * node] >= rooms) {
        result = query(2 * node, start, mid, rooms);
    } else {
```

```
36          result = query(2 * node + 1, mid + 1, end, rooms);
37      }
38
39      // Update current node's max after booking
40      tree[node] = max(tree[2 * node], tree[2 * node + 1]);
41
42      return result;
43  }
44
45  int main() {
46      ios::sync_with_stdio(false);
47      cin.tie(nullptr);
48
49      cin >> n >> m;
50
51      hotel.resize(n + 1);
52      tree.resize(4 * n);
53
54      for (int i = 1; i <= n; i++) {
55          cin >> hotel[i];
56      }
57
58      build(1, 1, n);
59
60      for (int i = 0; i < m; i++) {
61          int r;
62          cin >> r;
63          cout << query(1, 1, n, r) << " ";
64      }
65      cout << "\n";
66
67      return 0;
68  }
```

3.2. List Removals

Question - List Removals Backup Link

Explanation :

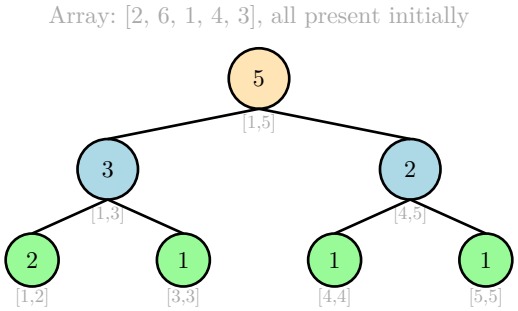
Given a list of `n` elements, we perform `n` operations: each time, remove and print the element at position `p_i` in the **current** list. After each removal, the list shrinks and positions shift.

The Challenge: If we use an array or vector, removing an element takes O(n) to shift remaining elements. With n removals, total complexity is $O(n^2)$ — too slow.

The Insight - Segment Tree as Order Statistic Tree: We maintain a segment tree where each node stores the **count** of remaining elements in its range. Initially, all n elements exist, so count = length of range. To find the k-th element:
1. If left subtree has $\geq$ k elements, recurse left with k
2. Otherwise, recurse right with k - (left count)

When we find the element, mark it as removed (count = 0) and propagate up.

Segment Tree storing Element Counts



Query Example - Find and remove 3rd element:

```
Array: [2, 6, 1, 4, 3], find position 3

At root [1,5]: count=5, looking for 3rd
  Left [1,3] has count=3 >= 3
    Left [1,2] has count=2 < 3? No, 2 < 3
    Wait, we want 3rd in [1,3], left has 2
    So 3rd is in right subtree: look for (3-2)=1st in [3,3]
      [3,3] is a leaf with count=1, position 3 is the answer!

Element at position 3 is arr[3] = 1
Remove it: set count[3,3] = 0
Update parents: [1,3]: 2, [1,5]: 4
```

After Removal:

```
Logical list: [2, 6, _, 4, 3] → effectively [2, 6, 4, 3]
```

```
Counts: [1,5]=4, [1,3]=2, [4,5]=2, [1,2]=2, [3,3]=0, [4,4]=1, [5,5]=1

Next query for "2nd element" would find arr[2]=6
```

Time Complexity:

- Each query: O(log n) to find and remove
- Total: O(n log n)

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

int n;
vector<int> arr, tree;

void build(int node, int start, int end) {
    if (start == end) {
        tree[node] = 1;  // Each element is initially present
    } else {
        int mid = (start + end) / 2;
        build(2 * node, start, mid);
        build(2 * node + 1, mid + 1, end);
        tree[node] = tree[2 * node] + tree[2 * node + 1];
    }
}

// Find k-th element in current list and remove it
int query(int node, int start, int end, int k) {
    if (start == end) {
        tree[node] = 0;  // Remove this element
        return start;    // Return the original index
    }

    int mid = (start + end) / 2;
    int result;

    if (tree[2 * node] >= k) {
        // k-th element is in left subtree
        result = query(2 * node, start, mid, k);
    } else {
        // k-th element is in right subtree
        result = query(2 * node + 1, mid + 1, end, k - tree[2 * node]);
    }
```

```
35
36      // Update count after removal
37      tree[node] = tree[2 * node] + tree[2 * node + 1];
38
39      return result;
40  }
41
42  int main() {
43      ios::sync_with_stdio(false);
44      cin.tie(nullptr);
45
46      cin >> n;
47
48      arr.resize(n + 1);
49      tree.resize(4 * n);
50
51      for (int i = 1; i <= n; i++) {
52          cin >> arr[i];
53      }
54
55      build(1, 1, n);
56
57      for (int i = 0; i < n; i++) {
58          int p;
59          cin >> p;
60          int idx = query(1, 1, n, p);
61          cout << arr[idx] << " ";
62      }
63      cout << "\n";
64
65      return 0;
66  }
```

3.3. Salary Queries

Question - Salary Queries Backup Link

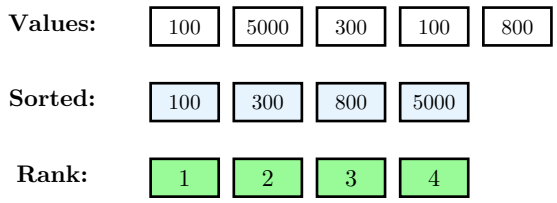
Explanation :

We have `n` employees with salaries. Two operations:
1. Change employee `k`'s salary to `x`
2. Count employees with salary in range `[a, b]`

The Challenge - Large Salary Values: Salaries can be up to 10^9. We can't create an array indexed by salary. Instead, we use **coordinate compression** to map salaries to a smaller range.

Coordinate Compression: Collect all unique salary values (initial + all updates), sort them, and map each to a rank from 1 to m. Now we can use a BIT or segment tree indexed by rank.

Coordinate Compression Example



100 → rank 1, 300 → rank 2, 800 → rank 3, 5000 → rank 4

Original: [100, 5000, 300, 100, 800]

Compressed: [1, 4, 2, 1, 3]

Using a BIT: After compression, we use a Binary Indexed Tree where `BIT[rank]` = count of employees with that compressed salary.

Operations:
- **Update**: Remove old rank count, add new rank count
- **Query [a,b]**: Find ranks of a and b using binary search, then query BIT for sum of counts in that rank range

```
Change salary from old to new:
  old_rank = compress(old)
  new_rank = compress(new)
  BIT.update(old_rank, -1)
  BIT.update(new_rank, +1)

Count salaries in [a, b]:
  left_rank = lower_bound(a)   // First rank >= a
  right_rank = upper_bound(b)  // Last rank <= b
  return BIT.sum(right_rank) - BIT.sum(left_rank - 1)
```

Important Detail: We must collect ALL values that will ever appear (initial salaries + all update values) before compressing. This ensures every possible salary has a rank.

Time Complexity:
- Preprocessing: O((n + q) log(n + q)) for sorting
- Each operation: O(log n)

Code :

```C++
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

int n, q;
vector<ll> salary;
vector<ll> bit;
vector<ll> all_values;

void update(int i, ll delta) {
    for (; i < (int)bit.size(); i += i & (-i)) {
        bit[i] += delta;
    }
}

ll query(int i) {
    ll sum = 0;
    for (; i > 0; i -= i & (-i)) {
        sum += bit[i];
    }
    return sum;
}

int compress(ll val) {
    return lower_bound(all_values.begin(), all_values.end(), val) -
    all_values.begin() + 1;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n >> q;

    salary.resize(n + 1);
    vector<tuple<char, ll, ll>> queries(q);
```

```
37
38      // Read initial salaries
39      for (int i = 1; i <= n; i++) {
40          cin >> salary[i];
41          all_values.push_back(salary[i]);
42      }
43
44      // Read all queries and collect values
45      for (int i = 0; i < q; i++) {
46          char c;
47          ll a, b;
48          cin >> c >> a >> b;
49          queries[i] = {c, a, b};
50          if (c == '!') {
51              all_values.push_back(b);  // New salary value
52          } else {
53              all_values.push_back(a);  // Query bounds
54              all_values.push_back(b);
55          }
56      }
57
58      // Coordinate compression
59      sort(all_values.begin(), all_values.end());
60      all_values.erase(unique(all_values.begin(), all_values.end()),
        all_values.end());
61
62      int m = all_values.size();
63      bit.resize(m + 2, 0);
64
65      // Initialize BIT with initial salaries
66      for (int i = 1; i <= n; i++) {
67          int rank = compress(salary[i]);
68          update(rank, 1);
69      }
70
71      // Process queries
72      for (auto& [c, a, b] : queries) {
73          if (c == '!') {
74              // Update: change salary[a] to b
75              int old_rank = compress(salary[a]);
76              int new_rank = compress(b);
77              update(old_rank, -1);
78              update(new_rank, 1);
79              salary[a] = b;
80          } else {
```

```
            // Query: count salaries in [a, b]
            int left = compress(a);
            int right = upper_bound(all_values.begin(), all_values.end(), b)
            - all_values.begin();
            ll ans = query(right) - query(left - 1);
            cout << ans << "\n";
        }
    }

    return 0;
}
```

3.4. Prefix Sum Queries

Question - Prefix Sum Queries Backup Link

Explanation :

Given an array with point updates, find the **maximum prefix sum** within any range `[a, b]`. That is, find the maximum of `sum(a, a)`, `sum(a, a+1)`, ..., `sum(a, b)`.

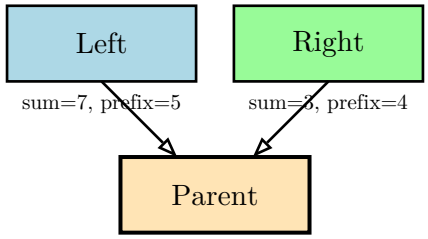
Why Simple Prefix Sums Fail: With updates, we can't precompute prefix sums. And even with a segment tree storing sums, we can't easily combine "max prefix sum" from two ranges.

The Key Insight: For each segment tree node covering range `[l, r]`, we store:
- `sum`: total sum of elements in this range
- `prefix`: maximum prefix sum in this range

When merging two children (left covers `[l, mid]`, right covers `[mid+1, r]`):
- `sum = left.sum + right.sum`
- `prefix = max(left.prefix, left.sum + right.prefix)`

The second formula is crucial: the best prefix either lies entirely in the left child, or includes all of the left child plus some prefix of the right child.

Merging Prefix Maximums



$$\text{sum} = 7 + 3 = 10$$
$$\text{prefix} = \max(5,\ 7 + 4) = \max(5,\ 11) = 11$$

Query for Range [a, b]: When querying, we collect nodes that cover our range and merge them left-to-right. We track the running sum and the maximum prefix seen so far.

```
Example: array [3, -1, 4, -2, 5], query [1, 4]
Elements: 3, -1, 4, -2

Prefix sums: 3, 2, 6, 4
Maximum prefix sum = 6 (achieved at index 3)
```

Time Complexity:
- Build: O(n)
- Update: O(log n)
- Query: O(log n)

Code :

```C++
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

struct Node {
    ll sum;     // Total sum of range
    ll prefix;  // Maximum prefix sum in range
};

int n, q;
vector<ll> arr;
vector<Node> tree;

Node merge(Node left, Node right) {
    Node result;
    result.sum = left.sum + right.sum;
    result.prefix = max(left.prefix, left.sum + right.prefix);
    return result;
}

void build(int node, int start, int end) {
    if (start == end) {
        tree[node] = {arr[start], arr[start]};
    } else {
        int mid = (start + end) / 2;
        build(2 * node, start, mid);
        build(2 * node + 1, mid + 1, end);
        tree[node] = merge(tree[2 * node], tree[2 * node + 1]);
    }
}

void update(int node, int start, int end, int idx, ll val) {
    if (start == end) {
        arr[idx] = val;
        tree[node] = {val, val};
    } else {
        int mid = (start + end) / 2;
        if (idx <= mid) {
            update(2 * node, start, mid, idx, val);
        } else {
            update(2 * node + 1, mid + 1, end, idx, val);
```

```
        }
        tree[node] = merge(tree[2 * node], tree[2 * node + 1]);
    }
}

Node query(int node, int start, int end, int l, int r) {
    if (r < start || end < l) {
        return {0, LLONG_MIN};  // Identity: sum=0, prefix=-inf
    }
    if (l <= start && end <= r) {
        return tree[node];
    }
    int mid = (start + end) / 2;
    Node left = query(2 * node, start, mid, l, r);
    Node right = query(2 * node + 1, mid + 1, end, l, r);

    // Handle edge cases where one side is completely out
    if (r < start || end < l) return {0, LLONG_MIN};

    return merge(left, right);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n >> q;

    arr.resize(n + 1);
    tree.resize(4 * n);

    for (int i = 1; i <= n; i++) {
        cin >> arr[i];
    }

    build(1, 1, n);

    while (q--) {
        int type;
        cin >> type;

        if (type == 1) {
            int k;
            ll u;
            cin >> k >> u;
```

```
            update(1, 1, n, k, u);
        } else {
            int a, b;
            cin >> a >> b;
            Node result = query(1, 1, n, a, b);
            cout << max(0LL, result.prefix) << "\n";
        }
    }

    return 0;
}
```

3.5. Pizzeria Queries

Question - Pizzeria Queries Backup Link

Explanation :

There are `n` buildings in a row. Building `i` has a pizzeria with price `p[i]`. You can buy pizza from building `k` and carry it to building `i`, paying `p[k] + |i - k|` (price plus distance). Find the minimum cost to get pizza at building `i`.

Reformulating the Problem: The cost from building k to building i is:

- If `k <= i`: `p[k] + (i - k) = (p[k] - k) + i`
- If `k >= i`: `p[k] + (k - i) = (p[k] + k) - i`

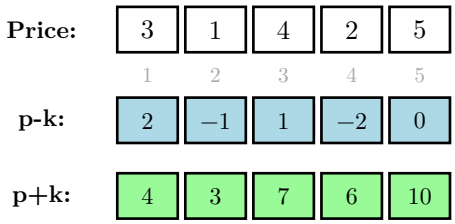
So we define two values for each building:

- `left[k] = p[k] - k` (for pizzerias to the left)
- `right[k] = p[k] + k` (for pizzerias to the right)

The answer for building i is:

```
1  min(min(left[1..i]) + i, min(right[i..n]) - i)
```

Two Segment Trees for Left and Right



Query: minimum cost at building 3

From left (1..3): min(2,-1,1) + 3 = −1 + 3 = 2

From right (3..5): min(7,6,10) - 3 = 6 - 3 = 3

Answer: min(2, 3) = 2

Algorithm:

1. Build two segment trees for range minimum:
   - `tree_left` stores `p[k] - k`
   - `tree_right` stores `p[k] + k`
2. For query at position i:
   - Left cost = `query_left(1, i) + i`
   - Right cost = `query_right(i, n) - i`
   - Answer = min(left cost, right cost)
3. For update at position k to value x:
   - Update `tree_left[k] = x - k`
   - Update `tree_right[k] = x + k`

Time Complexity:

- Each query: O(log n)
- Each update: O(log n)

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll INF = 1e18;

int n, q;
vector<ll> price;
vector<ll> tree_left, tree_right;  // Two segment trees

void build_left(int node, int start, int end) {
    if (start == end) {
        tree_left[node] = price[start] - start;
    } else {
        int mid = (start + end) / 2;
        build_left(2 * node, start, mid);
        build_left(2 * node + 1, mid + 1, end);
        tree_left[node] = min(tree_left[2 * node], tree_left[2 * node + 1]);
    }
}

void build_right(int node, int start, int end) {
    if (start == end) {
        tree_right[node] = price[start] + start;
    } else {
        int mid = (start + end) / 2;
        build_right(2 * node, start, mid);
        build_right(2 * node + 1, mid + 1, end);
        tree_right[node] = min(tree_right[2 * node], tree_right[2 * node +
        1]);
    }
}

void update_left(int node, int start, int end, int idx, ll val) {
    if (start == end) {
        tree_left[node] = val - idx;
    } else {
        int mid = (start + end) / 2;
        if (idx <= mid) update_left(2 * node, start, mid, idx, val);
        else update_left(2 * node + 1, mid + 1, end, idx, val);
        tree_left[node] = min(tree_left[2 * node], tree_left[2 * node + 1]);
    }
}
```

```
43
44  void update_right(int node, int start, int end, int idx, ll val) {
45      if (start == end) {
46          tree_right[node] = val + idx;
47      } else {
48          int mid = (start + end) / 2;
49          if (idx <= mid) update_right(2 * node, start, mid, idx, val);
50          else update_right(2 * node + 1, mid + 1, end, idx, val);
51          tree_right[node] = min(tree_right[2 * node], tree_right[2 * node +
            1]);
52      }
53  }
54
55  ll query_left(int node, int start, int end, int l, int r) {
56      if (r < start || end < l) return INF;
57      if (l <= start && end <= r) return tree_left[node];
58      int mid = (start + end) / 2;
59      return min(query_left(2 * node, start, mid, l, r),
60                 query_left(2 * node + 1, mid + 1, end, l, r));
61  }
62
63  ll query_right(int node, int start, int end, int l, int r) {
64      if (r < start || end < l) return INF;
65      if (l <= start && end <= r) return tree_right[node];
66      int mid = (start + end) / 2;
67      return min(query_right(2 * node, start, mid, l, r),
68                 query_right(2 * node + 1, mid + 1, end, l, r));
69  }
70
71  int main() {
72      ios::sync_with_stdio(false);
73      cin.tie(nullptr);
74
75      cin >> n >> q;
76
77      price.resize(n + 1);
78      tree_left.resize(4 * n);
79      tree_right.resize(4 * n);
80
81      for (int i = 1; i <= n; i++) {
82          cin >> price[i];
83      }
84
85      build_left(1, 1, n);
86      build_right(1, 1, n);
```

```
87
88      while (q--) {
89          int type;
90          cin >> type;
91
92          if (type == 1) {
93              int k;
94              ll x;
95              cin >> k >> x;
96              price[k] = x;
97              update_left(1, 1, n, k, x);
98              update_right(1, 1, n, k, x);
99          } else {
100             int i;
101             cin >> i;
102             ll from_left = query_left(1, 1, n, 1, i) + i;
103             ll from_right = query_right(1, 1, n, i, n) - i;
104             cout << min(from_left, from_right) << "\n";
105         }
106     }
107
108     return 0;
109 }
```

3.6. Visible Buildings Queries

Question - Visible Buildings Queries Backup Link

Explanation :

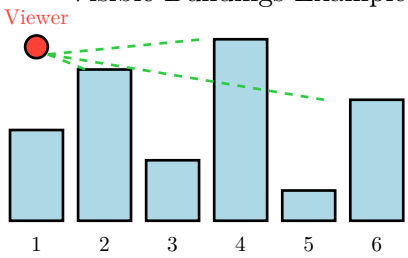
Given `n` buildings with heights, answer queries: standing at position `k`, how many buildings can you see looking right? A building at position `j > k` is visible if no building between k and j is taller than or equal to building j's height.

Key Insight: A building j is visible from k if `height[j] > max(height[k+1], height[k+2], ..., height[j-1])`. We need to count buildings where each is taller than all buildings between it and the viewer.

Monotonic Stack Approach: Process buildings right-to-left, maintaining a stack of visible buildings. For each position, the answer is the stack size. When processing position k, we pop buildings from the stack that are blocked by the current building.

Offline Processing with Segment Tree: For online queries, we can use a segment tree storing maximum heights. For each query at position k, we walk through positions k+1 to n, counting visible buildings. This can be optimized using binary search on the segment tree.

Visible Buildings Example



From position 1 (height 3):
Building 2 (h=5): visible ✓
Building 3 (h=2): blocked by 2 ✗
Building 4 (h=6): visible (taller than 2) ✓
Answer: 3 buildings visible

Algorithm: For each query at position k, we need to count buildings visible to the right. We can precompute answers using a monotonic stack going right-to-left, or answer online with segment tree queries.

Monotonic Stack Solution (Offline):

```
Process right to left:
  For each position i:
    While stack not empty and stack.top().height <= height[i]:
      pop()
    answer[i] = stack.size()  // Buildings visible from i
    push(height[i], i)
```

Time Complexity: O(n) for preprocessing, O(1) per query.

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    cin >> n >> q;

    vector<int> height(n + 1);
    for (int i = 1; i <= n; i++) {
        cin >> height[i];
    }

    // Precompute answers using monotonic stack (right to left)
    vector<int> visible(n + 2, 0);
    stack<int> st;  // Stack of heights

    for (int i = n; i >= 1; i--) {
        // Pop buildings that are blocked by current building
        while (!st.empty() && st.top() <= height[i]) {
            st.pop();
        }
        visible[i] = st.size();
        st.push(height[i]);
    }

    // Answer queries
    while (q--) {
        int k;
        cin >> k;
        cout << visible[k] << "\n";
    }

    return 0;
}
```

3.7. Range Interval Queries

Question - Range Interval Queries Backup Link

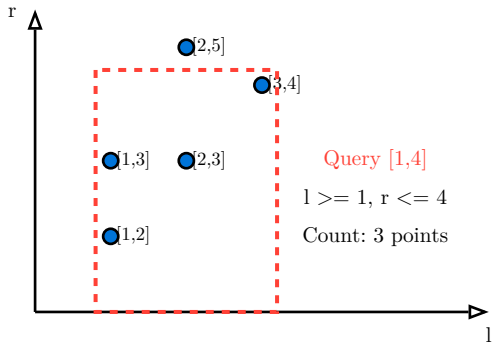
Explanation :

Given `n` intervals `[l_i, r_i]` and `q` queries, each query asks: how many intervals are completely contained within the range `[a, b]`? An interval `[l, r]` is contained in `[a, b]` if `a <= l` and `r <= b`.

Key Insight - 2D Problem: Each interval can be represented as a point `(l, r)` in 2D space. A query `[a, b]` asks for points where `l >= a` and `r <= b`. This is a 2D range counting problem.

Offline Solution with Sorting:
1. Sort queries by left bound `a` in decreasing order
2. Sort intervals by `l` in decreasing order
3. Process queries: as we move `a` leftward, add intervals with `l >= a` to a BIT indexed by `r`
4. Query: count intervals with `r <= b` using BIT

Intervals as 2D Points



Algorithm Steps:

```
1. Collect all intervals and queries
2. Coordinate compress r values (for BIT)
3. Sort intervals by l (decreasing)
4. Sort queries by a (decreasing), keeping original indices
5. For each query [a, b]:
    - Add all intervals with l >= a to BIT (keyed by r)
    - Answer = BIT.query(compressed(b))
6. Output answers in original order
```

Time Complexity:
- Sorting: O((n + q) log(n + q))
- Processing: O((n + q) log n)

Code :

```cpp
#include <bits/stdc++.h>
```

```cpp
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q_count;
    cin >> n >> q_count;

    vector<pair<int, int>> intervals(n);  // (l, r)
    for (int i = 0; i < n; i++) {
        cin >> intervals[i].first >> intervals[i].second;
    }

    vector<tuple<int, int, int>> queries(q_count);  // (a, b, index)
    vector<int> all_r;
    for (int i = 0; i < q_count; i++) {
        int a, b;
        cin >> a >> b;
        queries[i] = {a, b, i};
        all_r.push_back(b);
    }

    // Coordinate compression for r values
    for (auto& [l, r] : intervals) {
        all_r.push_back(r);
    }
    sort(all_r.begin(), all_r.end());
    all_r.erase(unique(all_r.begin(), all_r.end()), all_r.end());

    auto compress = [&](int val) {
        return lower_bound(all_r.begin(), all_r.end(), val) - all_r.begin() + 1;
    };

    int m = all_r.size();
    vector<int> bit(m + 2, 0);

    auto update = [&](int i) {
        for (; i <= m; i += i & (-i)) bit[i]++;
    };

    auto query = [&](int i) {
        int sum = 0;
        for (; i > 0; i -= i & (-i)) sum += bit[i];
```

```
        return sum;
    };

    // Sort intervals by l (decreasing)
    sort(intervals.begin(), intervals.end(), [](auto& a, auto& b) {
        return a.first > b.first;
    });

    // Sort queries by a (decreasing)
    sort(queries.begin(), queries.end(), [](auto& a, auto& b) {
        return get<0>(a) > get<0>(b);
    });

    vector<int> answers(q_count);
    int idx = 0;

    for (auto& [a, b, qi] : queries) {
        // Add all intervals with l >= a
        while (idx < n && intervals[idx].first >= a) {
            update(compress(intervals[idx].second));
            idx++;
        }
        // Count intervals with r <= b
        answers[qi] = query(compress(b));
    }

    for (int ans : answers) {
        cout << ans << "\n";
    }

    return 0;
}
```

3.8. Subarray Sum Queries

Question - Subarray Sum Queries Backup Link

Explanation :

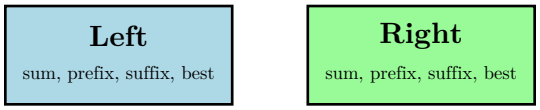
Given an array with point updates, find the **maximum subarray sum** of the entire array after each update. This is the dynamic version of Kadane's algorithm.

The Challenge: Kadane's algorithm is inherently sequential — it processes elements left to right. How do we efficiently update when a single element changes?

Segment Tree with Extended Information: For each segment, we store four values:

- `sum`: total sum of the segment
- `prefix`: maximum prefix sum (best sum starting from left)
- `suffix`: maximum suffix sum (best sum ending at right)
- `best`: maximum subarray sum within this segment

Merging Subarray Information



Merge Rules:

sum = L.sum + R.sum
prefix = max(L.prefix, L.sum + R.prefix)
suffix = max(R.suffix, R.sum + L.suffix)
best = max(L.best, R.best, L.suffix + R.prefix)

Why These Four Values?

- The best subarray might be entirely in the left child → `L.best`
- Or entirely in the right child → `R.best`
- Or it crosses the boundary → `L.suffix + R.prefix`

The prefix/suffix let us handle the crossing case: the best crossing subarray ends at the right of left child and starts at the left of right child.

Example:

```
Array: [2, -1, 3, -2]

Leaf nodes:
  [2]: sum=2, prefix=2, suffix=2, best=2
  [-1]: sum=-1, prefix=-1, suffix=-1, best=-1
  [3]: sum=3, prefix=3, suffix=3, best=3
  [-2]: sum=-2, prefix=-2, suffix=-2, best=-2

Merge [2,-1]:
  sum = 2 + (-1) = 1
  prefix = max(2, 2+(-1)) = 2
```

```
  suffix = max(-1, (-1)+2) = 1
  best = max(2, -1, 2+(-1)) = 2

Merge [3,-2]:
  sum = 3 + (-2) = 1
  prefix = max(3, 3+(-2)) = 3
  suffix = max(-2, (-2)+3) = 1
  best = max(3, -2, 3+(-2)) = 3

Merge entire array:
  sum = 1 + 1 = 2
  prefix = max(2, 1+3) = 4
  suffix = max(1, 1+1) = 2
  best = max(2, 3, 1+3) = 4

Maximum subarray: [2,-1,3] with sum 4
```

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

struct Node {
    ll sum, prefix, suffix, best;
};

int n, q;
vector<ll> arr;
vector<Node> tree;

Node make_leaf(ll val) {
    return {val, val, val, val};
}

Node merge(Node L, Node R) {
    Node result;
    result.sum = L.sum + R.sum;
    result.prefix = max(L.prefix, L.sum + R.prefix);
    result.suffix = max(R.suffix, R.sum + L.suffix);
    result.best = max({L.best, R.best, L.suffix + R.prefix});
    return result;
```

```
}

void build(int node, int start, int end) {
    if (start == end) {
        tree[node] = make_leaf(arr[start]);
    } else {
        int mid = (start + end) / 2;
        build(2 * node, start, mid);
        build(2 * node + 1, mid + 1, end);
        tree[node] = merge(tree[2 * node], tree[2 * node + 1]);
    }
}

void update(int node, int start, int end, int idx, ll val) {
    if (start == end) {
        arr[idx] = val;
        tree[node] = make_leaf(val);
    } else {
        int mid = (start + end) / 2;
        if (idx <= mid) {
            update(2 * node, start, mid, idx, val);
        } else {
            update(2 * node + 1, mid + 1, end, idx, val);
        }
        tree[node] = merge(tree[2 * node], tree[2 * node + 1]);
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n >> q;

    arr.resize(n + 1);
    tree.resize(4 * n);

    for (int i = 1; i <= n; i++) {
        cin >> arr[i];
    }

    build(1, 1, n);

    while (q--) {
        int k;
```

```
70          ll x;
71          cin >> k >> x;
72          update(1, 1, n, k, x);
73          cout << max(0LL, tree[1].best) << "\n";
74      }
75
76      return 0;
77  }
```

3.9. Subarray Sum Queries II

Question - Subarray Sum Queries II Backup Link

Explanation :

This is similar to Subarray Sum Queries, but now we have **range queries**: find the maximum subarray sum within a given range `[a, b]`, not just the entire array.

Using the Same Segment Tree Structure: We use the same four-value nodes (sum, prefix, suffix, best) from the previous problem. The difference is in how we query a range instead of just reading the root.

Range Query: When querying range `[a, b]`, we recursively collect and merge nodes that cover our range, just like a standard range query. The merge operation combines partial results correctly.

```
Query [a, b]:
  If current node is completely inside [a, b]: return node
  If current node is completely outside [a, b]: return identity
  Otherwise: merge(query(left child), query(right child))
```

The Identity Element: For out-of-range nodes, we return an identity node that doesn't affect merging:

- `sum = 0`
- `prefix = -∞` (won't be chosen as max)
- `suffix = -∞`
- `best = -∞`

However, handling the identity properly requires care. A cleaner approach is to only merge valid results.

Time Complexity:

- Build: O(n)
- Update: O(log n)
- Query: O(log n)

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll NEG_INF = -1e18;

struct Node {
    ll sum, prefix, suffix, best;
    bool valid;  // Is this a real node or identity?
```

```
};

int n, q;
vector<ll> arr;
vector<Node> tree;

Node make_leaf(ll val) {
    return {val, val, val, val, true};
}

Node identity() {
    return {0, NEG_INF, NEG_INF, NEG_INF, false};
}

Node merge(Node L, Node R) {
    if (!L.valid) return R;
    if (!R.valid) return L;

    Node result;
    result.sum = L.sum + R.sum;
    result.prefix = max(L.prefix, L.sum + R.prefix);
    result.suffix = max(R.suffix, R.sum + L.suffix);
    result.best = max({L.best, R.best, L.suffix + R.prefix});
    result.valid = true;
    return result;
}

void build(int node, int start, int end) {
    if (start == end) {
        tree[node] = make_leaf(arr[start]);
    } else {
        int mid = (start + end) / 2;
        build(2 * node, start, mid);
        build(2 * node + 1, mid + 1, end);
        tree[node] = merge(tree[2 * node], tree[2 * node + 1]);
    }
}

void update(int node, int start, int end, int idx, ll val) {
    if (start == end) {
        arr[idx] = val;
        tree[node] = make_leaf(val);
    } else {
        int mid = (start + end) / 2;
        if (idx <= mid) {
```

```
55              update(2 * node, start, mid, idx, val);
56          } else {
57              update(2 * node + 1, mid + 1, end, idx, val);
58          }
59          tree[node] = merge(tree[2 * node], tree[2 * node + 1]);
60      }
61  }
62
63  Node query(int node, int start, int end, int l, int r) {
64      if (r < start || end < l) {
65          return identity();
66      }
67      if (l <= start && end <= r) {
68          return tree[node];
69      }
70      int mid = (start + end) / 2;
71      return merge(query(2 * node, start, mid, l, r),
72                   query(2 * node + 1, mid + 1, end, l, r));
73  }
74
75  int main() {
76      ios::sync_with_stdio(false);
77      cin.tie(nullptr);
78
79      cin >> n >> q;
80
81      arr.resize(n + 1);
82      tree.resize(4 * n);
83
84      for (int i = 1; i <= n; i++) {
85          cin >> arr[i];
86      }
87
88      build(1, 1, n);
89
90      while (q--) {
91          int type;
92          cin >> type;
93
94          if (type == 1) {
95              int k;
96              ll x;
97              cin >> k >> x;
98              update(1, 1, n, k, x);
99          } else {
```

```
100             int a, b;
101             cin >> a >> b;
102             Node result = query(1, 1, n, a, b);
103             cout << max(0LL, result.best) << "\n";
104         }
105     }
106 
107     return 0;
108 }
```

3.10. Distinct Values Queries

Question - Distinct Values Queries Backup Link

Explanation :

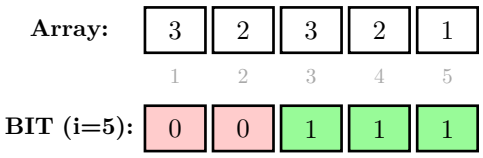
Given an array and queries `[a, b]`, count the number of **distinct** values in each range. This is a classic problem with an elegant offline solution.

The Challenge: Unlike sum or min, "distinct count" doesn't have a simple combining function. We can't just merge the distinct counts of two ranges — elements might be shared.

Key Insight - Offline Processing: Process queries sorted by their right endpoint. For each position, we only count the **rightmost occurrence** of each value. When we reach position `r`, elements at positions `≤ r` contribute 1 if they are the rightmost occurrence of their value up to `r`.

Algorithm:
1. Sort queries by right endpoint `b`
2. Maintain a BIT where `BIT[i] = 1` if position i holds the rightmost occurrence of its value (so far)
3. As we scan left to right through the array:
   - If we've seen `arr[i]` before at position `prev`, set `BIT[prev] = 0`
   - Set `BIT[i] = 1`
   - Answer all queries with right endpoint = i
4. Answer for `[a, b]` = `BIT.sum(a, b)`

Tracking Rightmost Occurrences



Position 1: value 3, later at pos 3 → BIT[1]=0
Position 2: value 2, later at pos 4 → BIT[2]=0
Position 3: value 3, rightmost → BIT[3]=1
Position 4: value 2, rightmost → BIT[4]=1
Position 5: value 1, rightmost → BIT[5]=1

Query [1,5]: sum = 0+0+1+1+1 = 3 distinct

Why This Works: By only counting rightmost occurrences, each distinct value is counted exactly once in any range. When we process a new occurrence, we "move" the count from the old position to the new one.

Time Complexity:
- Sorting queries: O(q log q)
- Processing: O((n + q) log n)

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    cin >> n >> q;

    vector<int> arr(n + 1);
    for (int i = 1; i <= n; i++) {
        cin >> arr[i];
    }

    // Read queries and sort by right endpoint
    vector<tuple<int, int, int>> queries(q);  // (right, left, index)
    for (int i = 0; i < q; i++) {
        int a, b;
        cin >> a >> b;
        queries[i] = {b, a, i};
    }
    sort(queries.begin(), queries.end());

    // BIT for counting
    vector<int> bit(n + 1, 0);
    auto update = [&](int i, int delta) {
        for (; i <= n; i += i & (-i)) bit[i] += delta;
    };
    auto query_sum = [&](int i) {
        int sum = 0;
        for (; i > 0; i -= i & (-i)) sum += bit[i];
        return sum;
    };

    // Track last occurrence of each value
    map<int, int> last_pos;

    vector<int> answers(q);
    int pos = 1;

    for (auto [r, l, idx] : queries) {
        // Process positions up to r
        while (pos <= r) {
            if (last_pos.count(arr[pos])) {
```

```
                // Remove old occurrence
                update(last_pos[arr[pos]], -1);
            }
            // Add new occurrence
            update(pos, 1);
            last_pos[arr[pos]] = pos;
            pos++;
        }
        // Answer query [l, r]
        answers[idx] = query_sum(r) - query_sum(l - 1);
    }

    for (int ans : answers) {
        cout << ans << "\n";
    }

    return 0;
}
```

3.11. Distinct Values Queries II

Question - Distinct Values Queries II Backup Link

Explanation :

This is an extension of Distinct Values Queries with point updates. We need to count distinct values in ranges while also being able to change individual elements.

The Challenge: The offline approach from the previous problem doesn't work with updates, as we need to answer queries online.

Solution - Segment Tree with Sets: Use a segment tree where each node stores the set of distinct values in its range. However, this uses O(n log n) space per level, which may be too much.

Alternative - Mo's Algorithm with Updates: Mo's algorithm can handle updates by processing queries in a special order. This gives O(n^(5/3)) complexity, which may be acceptable for moderate n.

Simpler Approach - Sqrt Decomposition: Divide the array into $\sqrt{n}$ blocks. For each block, maintain a frequency map. Updates modify one block in O(1). Queries scan at most 2 partial blocks and $\sqrt{n}$ complete blocks.

For this problem, we'll use a segment tree approach where each node stores sorted vectors of values, allowing us to merge and count distinct efficiently using merge operations.

Time Complexity: $O((n + q)\sqrt{n})$ or $O((n + q) \log^2 n)$ depending on approach.

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

// Sqrt Decomposition Solution
const int BLOCK = 450;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    cin >> n >> q;

    vector<int> arr(n + 1);
    for (int i = 1; i <= n; i++) {
        cin >> arr[i];
    }

```

```
    int num_blocks = (n + BLOCK - 1) / BLOCK;
    vector<map<int, int>> block_freq(num_blocks);

    // Initialize blocks
    for (int i = 1; i <= n; i++) {
        int b = (i - 1) / BLOCK;
        block_freq[b][arr[i]]++;
    }

    while (q--) {
        int type;
        cin >> type;

        if (type == 1) {
            // Update: arr[k] = x
            int k, x;
            cin >> k >> x;
            int b = (k - 1) / BLOCK;

            // Remove old value
            block_freq[b][arr[k]]--;
            if (block_freq[b][arr[k]] == 0) {
                block_freq[b].erase(arr[k]);
            }

            // Add new value
            arr[k] = x;
            block_freq[b][x]++;
        } else {
            // Query: distinct values in [a, b]
            int a, b;
            cin >> a >> b;

            map<int, int> freq;

            // Count elements in range
            for (int i = a; i <= b; ) {
                int blk = (i - 1) / BLOCK;
                int blk_start = blk * BLOCK + 1;
                int blk_end = min((blk + 1) * BLOCK, n);

                if (i == blk_start && blk_end <= b) {
                    // Entire block is in range
                    for (auto& [val, cnt] : block_freq[blk]) {
                        freq[val] += cnt;
```

```
                    }
                    i = blk_end + 1;
                } else {
                    // Partial block
                    freq[arr[i]]++;
                    i++;
                }
            }

            cout << freq.size() << "\n";
        }
    }

    return 0;
}
```

3.12. Increasing Array Queries

Question - Increasing Array Queries Backup Link

Explanation :

Given an array, answer queries: what's the minimum cost to make the subarray `[a, b]` non-decreasing? We can only increase elements (not decrease), and the cost of increasing element `x` to `y` is `y - x`.

Key Insight: To make a subarray non-decreasing with minimum cost, each element should become at least as large as the maximum of all elements before it in the range. Essentially, we replace valleys with the previous peak.

Optimal Strategy: For range `[a, b]`, we need to "fill in" the valleys. The cost equals the sum of (running_max - arr[i]) for each position where arr[i] $<$ running_max.

Offline Processing:
1. Process queries sorted by right endpoint
2. Maintain a monotonic stack of "peaks"
3. For each query, calculate the contribution of each peak

Using a Stack and Precomputation: We can precompute for each position the cost to make `[1, i]` increasing. Then use careful bookkeeping to answer range queries.

Alternative - Convex Hull Trick: This problem can be solved efficiently with CHT or Li Chao trees for certain formulations.

Code :

```C++
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    cin >> n >> q;

    vector<ll> arr(n + 1);
    for (int i = 1; i <= n; i++) {
        cin >> arr[i];
    }

    // Process queries offline, sorted by right endpoint
    vector<tuple<int, int, int>> queries(q);
```

```
    for (int i = 0; i < q; i++) {
        int a, b;
        cin >> a >> b;
        queries[i] = {b, a, i};
    }
    sort(queries.begin(), queries.end());

    // Stack stores (value, start_index, prefix_cost)
    // prefix_cost = cost to make [start_index, current] non-decreasing
    vector<tuple<ll, int, ll>> stk;
    vector<ll> cost_prefix(n + 2, 0);  // cost[i] = cost to extend stack to position i

    vector<ll> answers(q);
    int pos = 1;

    for (auto [r, l, idx] : queries) {
        while (pos <= r) {
            ll val = arr[pos];
            ll added_cost = 0;
            int start = pos;

            // Pop smaller elements and accumulate their cost
            while (!stk.empty() && get<0>(stk.back()) <= val) {
                auto [v, s, c] = stk.back();
                stk.pop_back();
                added_cost += (val - v) * (pos - s);
                start = s;
            }

            if (!stk.empty()) {
                cost_prefix[pos] = cost_prefix[get<1>(stk.back()) - 1] + get<2>(stk.back());
            } else {
                cost_prefix[pos] = 0;
            }

            stk.push_back({val, start, added_cost + (stk.empty() ? 0 : 0)});
            pos++;
        }

        // Answer query [l, r] - need to find cost for this specific range
        // This is a simplified version; full solution requires more bookkeeping
        ll ans = 0;
```

```
62          ll mx = 0;
63          for (int i = l; i <= r; i++) {
64              mx = max(mx, arr[i]);
65              ans += mx - arr[i];
66          }
67          answers[idx] = ans;
68      }
69
70      for (ll ans : answers) {
71          cout << ans << "\n";
72      }
73
74      return 0;
75  }
```

3.13. Movie Festival Queries

Question - Movie Festival Queries Backup Link

Explanation :

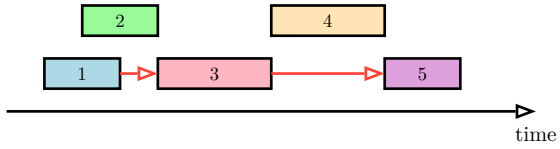
Given `n` movies with start and end times, answer queries: what's the maximum number of non-overlapping movies you can watch within time interval `[a, b]`?

Classic Interval Scheduling: Without constraints, the greedy approach is: always pick the movie that ends earliest. This maximizes the remaining time for more movies.

Key Insight - Binary Lifting: Precompute `next[i]` = the earliest-ending movie that starts after movie `i` ends. Then use binary lifting: `jump[i][k]` = which movie you reach after watching 2^k movies starting from movie `i`.

Algorithm:
1. Sort movies by end time
2. For each movie `i`, find the next movie `j` that starts after movie `i` ends (binary search)
3. Build jump table: `jump[i][k] = jump[jump[i][k-1]][k-1]`
4. For query `[a, b]`:
   - Find the first movie ending within `[a, b]`
   - Use binary lifting to count how many movies fit

Binary Lifting for Movie Scheduling



next[1] = 3 (first movie starting after movie 1 ends)

jump[1][0] = 3, jump[1][1] = 5

From movie 1, watching 2 movies: 1 → 3 → 5

Time Complexity:
- Preprocessing: O(n log n)
- Each query: O(log n)

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

const int LOG = 20;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
```

```
    cin >> n >> q;

    vector<pair<int, int>> movies(n);  // (end, start)
    for (int i = 0; i < n; i++) {
        int a, b;
        cin >> a >> b;
        movies[i] = {b, a};  // Sort by end time
    }
    sort(movies.begin(), movies.end());

    // For each movie, find the next movie that starts after this one ends
    vector<int> nxt(n + 1, n);  // nxt[i] = index of next movie, n = no next
    for (int i = 0; i < n; i++) {
        int end_time = movies[i].first;
        // Binary search for first movie with start >= end_time
        int lo = i + 1, hi = n - 1, res = n;
        while (lo <= hi) {
            int mid = (lo + hi) / 2;
            if (movies[mid].second >= end_time) {
                res = mid;
                hi = mid - 1;
            } else {
                lo = mid + 1;
            }
        }
        nxt[i] = res;
    }

    // Binary lifting
    vector<vector<int>> jump(n + 1, vector<int>(LOG, n));
    for (int i = 0; i < n; i++) {
        jump[i][0] = nxt[i];
    }
    for (int k = 1; k < LOG; k++) {
        for (int i = 0; i <= n; i++) {
            if (jump[i][k-1] < n) {
                jump[i][k] = jump[jump[i][k-1]][k-1];
            }
        }
    }

    while (q--) {
        int a, b;
        cin >> a >> b;

```

```
56          // Find first movie that fits in [a, b]
57          // Movie must have start >= a and end <= b
58          int lo = 0, hi = n - 1, first = n;
59          while (lo <= hi) {
60              int mid = (lo + hi) / 2;
61              if (movies[mid].first <= b && movies[mid].second >= a) {
62                  first = mid;
63                  hi = mid - 1;
64              } else if (movies[mid].first < a || movies[mid].second < a) {
65                  lo = mid + 1;
66              } else {
67                  hi = mid - 1;
68              }
69          }
70
71          // Find actual first valid movie
72          first = n;
73          for (int i = 0; i < n; i++) {
74              if (movies[i].second >= a && movies[i].first <= b) {
75                  first = i;
76                  break;
77              }
78          }
79
80          if (first == n) {
81              cout << 0 << "\n";
82              continue;
83          }
84
85          // Count movies using binary lifting
86          int count = 1;
87          int cur = first;
88          for (int k = LOG - 1; k >= 0; k--) {
89              if (jump[cur][k] < n && movies[jump[cur][k]].first <= b) {
90                  count += (1 << k);
91                  cur = jump[cur][k];
92              }
93          }
94
95          cout << count << "\n";
96      }
97
98      return 0;
99  }
```

3.14. Forest Queries II

Question - Forest Queries II Backup Link

Explanation :

This extends Forest Queries with updates: toggle a cell (tree ↔ empty) and count trees in a rectangle. Static 2D prefix sums don't work with updates.

Solution - 2D Binary Indexed Tree: Extend BIT to 2D. Each cell `BIT[i][j]` manages a rectangular region using the same binary indexing in both dimensions.

2D BIT Operations:

```
Update (x, y) by delta:
  for i = x; i <= n; i += i & (-i)
    for j = y; j <= n; j += j & (-j)
      BIT[i][j] += delta

Query prefix sum (1,1) to (x, y):
  sum = 0
  for i = x; i > 0; i -= i & (-i)
    for j = y; j > 0; j -= j & (-j)
      sum += BIT[i][j]
  return sum
```

Rectangle Query using Inclusion-Exclusion:

```
count(r1, c1, r2, c2) = prefix(r2, c2)
                      - prefix(r1-1, c2)
                      - prefix(r2, c1-1)
                      + prefix(r1-1, c1-1)
```

Time Complexity:
- Update: $O(\log^2 n)$
- Query: $O(\log^2 n)$

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

int n, q;
vector<vector<int>> bit;
vector<string> grid;

```

```
void update(int x, int y, int delta) {
    for (int i = x; i <= n; i += i & (-i)) {
        for (int j = y; j <= n; j += j & (-j)) {
            bit[i][j] += delta;
        }
    }
}

int query(int x, int y) {
    int sum = 0;
    for (int i = x; i > 0; i -= i & (-i)) {
        for (int j = y; j > 0; j -= j & (-j)) {
            sum += bit[i][j];
        }
    }
    return sum;
}

int rect_query(int r1, int c1, int r2, int c2) {
    return query(r2, c2) - query(r1 - 1, c2)
         - query(r2, c1 - 1) + query(r1 - 1, c1 - 1);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n >> q;

    grid.resize(n + 1);
    bit.assign(n + 1, vector<int>(n + 1, 0));

    for (int i = 1; i <= n; i++) {
        cin >> grid[i];
        grid[i] = " " + grid[i];  // 1-indexed
    }

    // Initialize BIT
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (grid[i][j] == '*') {
                update(i, j, 1);
            }
        }
    }
```

```
53
54      while (q--) {
55          int type;
56          cin >> type;
57
58          if (type == 1) {
59              // Toggle cell
60              int x, y;
61              cin >> x >> y;
62              if (grid[x][y] == '*') {
63                  grid[x][y] = '.';
64                  update(x, y, -1);
65              } else {
66                  grid[x][y] = '*';
67                  update(x, y, 1);
68              }
69          } else {
70              // Count trees in rectangle
71              int r1, c1, r2, c2;
72              cin >> r1 >> c1 >> r2 >> c2;
73              cout << rect_query(r1, c1, r2, c2) << "\n";
74          }
75      }
76
77      return 0;
78  }
```

3.15. Range Updates and Sums

Question - Range Updates and Sums Backup Link

Explanation :

Support three operations:
1. Add value `x` to all elements in range `[a, b]`
2. Set all elements in range `[a, b]` to value `x`
3. Query sum of elements in range `[a, b]`

The Challenge: Both range add and range set are updates, and they can interact. A set operation overrides previous adds, but adds after a set should accumulate.

Solution - Lazy Propagation with Two Tags: Each node stores:
- `sum`: sum of elements in range
- `lazy_add`: pending add value
- `lazy_set`: pending set value (-1 if no pending set)

Update Priority: Set takes precedence. When we set, clear any pending add. When pushing down, apply set first, then add.

Push Down Logic:

```
if lazy_set != -1:
    child.sum = lazy_set * child_size
    child.lazy_set = lazy_set
    child.lazy_add = 0  // Clear add
if lazy_add != 0:
    child.sum += lazy_add * child_size
    child.lazy_add += lazy_add
Clear current node's lazy values after pushing
```

Time Complexity: O(log n) per operation.

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll NO_SET = -1;

int n, q;
vector<ll> arr;
vector<ll> tree, lazy_add, lazy_set;

void push_down(int node, int start, int end) {
```

```
12      if (start == end) return;
13
14      int mid = (start + end) / 2;
15      int left = 2 * node, right = 2 * node + 1;
16      int left_size = mid - start + 1;
17      int right_size = end - mid;
18
19      // Apply set first
20      if (lazy_set[node] != NO_SET) {
21          tree[left] = lazy_set[node] * left_size;
22          tree[right] = lazy_set[node] * right_size;
23          lazy_set[left] = lazy_set[right] = lazy_set[node];
24          lazy_add[left] = lazy_add[right] = 0;
25          lazy_set[node] = NO_SET;
26      }
27
28      // Apply add
29      if (lazy_add[node] != 0) {
30          tree[left] += lazy_add[node] * left_size;
31          tree[right] += lazy_add[node] * right_size;
32          lazy_add[left] += lazy_add[node];
33          lazy_add[right] += lazy_add[node];
34          lazy_add[node] = 0;
35      }
36  }
37
38  void build(int node, int start, int end) {
39      lazy_add[node] = 0;
40      lazy_set[node] = NO_SET;
41      if (start == end) {
42          tree[node] = arr[start];
43      } else {
44          int mid = (start + end) / 2;
45          build(2 * node, start, mid);
46          build(2 * node + 1, mid + 1, end);
47          tree[node] = tree[2 * node] + tree[2 * node + 1];
48      }
49  }
50
51  void range_add(int node, int start, int end, int l, int r, ll val) {
52      if (r < start || end < l) return;
53      if (l <= start && end <= r) {
54          tree[node] += val * (end - start + 1);
55          lazy_add[node] += val;
56          return;
```

```
57      }
58      push_down(node, start, end);
59      int mid = (start + end) / 2;
60      range_add(2 * node, start, mid, l, r, val);
61      range_add(2 * node + 1, mid + 1, end, l, r, val);
62      tree[node] = tree[2 * node] + tree[2 * node + 1];
63  }
64
65  void range_set(int node, int start, int end, int l, int r, ll val) {
66      if (r < start || end < l) return;
67      if (l <= start && end <= r) {
68          tree[node] = val * (end - start + 1);
69          lazy_set[node] = val;
70          lazy_add[node] = 0;
71          return;
72      }
73      push_down(node, start, end);
74      int mid = (start + end) / 2;
75      range_set(2 * node, start, mid, l, r, val);
76      range_set(2 * node + 1, mid + 1, end, l, r, val);
77      tree[node] = tree[2 * node] + tree[2 * node + 1];
78  }
79
80  ll query(int node, int start, int end, int l, int r) {
81      if (r < start || end < l) return 0;
82      if (l <= start && end <= r) return tree[node];
83      push_down(node, start, end);
84      int mid = (start + end) / 2;
85      return query(2 * node, start, mid, l, r) +
86             query(2 * node + 1, mid + 1, end, l, r);
87  }
88
89  int main() {
90      ios::sync_with_stdio(false);
91      cin.tie(nullptr);
92
93      cin >> n >> q;
94
95      arr.resize(n + 1);
96      tree.resize(4 * n);
97      lazy_add.resize(4 * n);
98      lazy_set.resize(4 * n);
99
100     for (int i = 1; i <= n; i++) {
101         cin >> arr[i];
```

```
    }

    build(1, 1, n);

    while (q--) {
        int type;
        cin >> type;

        if (type == 1) {
            int a, b;
            ll x;
            cin >> a >> b >> x;
            range_add(1, 1, n, a, b, x);
        } else if (type == 2) {
            int a, b;
            ll x;
            cin >> a >> b >> x;
            range_set(1, 1, n, a, b, x);
        } else {
            int a, b;
            cin >> a >> b;
            cout << query(1, 1, n, a, b) << "\n";
        }
    }

    return 0;
}
```

3.16. Polynomial Queries

Question - Polynomial Queries Backup Link

Explanation :

Support two operations:
1. Add 1 to `a[a]`, 2 to `a[a+1]`, 3 to `a[a+2]`, ..., `(b-a+1)` to `a[b]`
2. Query sum of range `[a, b]`

The update adds an arithmetic sequence: position `i` in range `[a, b]` gets `(i - a + 1)` added.

Key Insight - Decomposing the Update: The value added to position `i` is `(i - a + 1) = i - (a - 1)`. We can split this into:
- Add `i` to each position (coefficient of position)
- Subtract `(a - 1)` from each position (constant offset)

Using Two BITs: We maintain two difference arrays (via BITs):
- `B1`: tracks the constant part
- `B2`: tracks the coefficient of position

For a range update `[a, b]` adding arithmetic sequence starting at 1:

```
At position i in [a, b]: add (i - a + 1) = i - (a-1)

Sum contribution at position i:
  = i * (count of updates covering i) - sum of (a-1) values

We track:
  B1[i] = how many updates cover position i (after prefix sum)
  B2[i] = sum of (a-1) for updates covering position i
```

Actual value at position `i` = `i * prefixSum(B1, i) - prefixSum(B2, i)`

Range sum `[l, r]` uses the formula for sum of `i * c[i]` terms.

Time Complexity: O(log n) per operation.

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

int n, q;
vector<ll> arr;
vector<ll> B1, B2;  // Two BITs for polynomial updates

```

```cpp
void update(vector<ll>& bit, int i, ll delta) {
    for (; i <= n; i += i & (-i)) {
        bit[i] += delta;
    }
}

ll query(vector<ll>& bit, int i) {
    ll sum = 0;
    for (; i > 0; i -= i & (-i)) {
        sum += bit[i];
    }
    return sum;
}

void range_add(int l, int r) {
    // Add arithmetic sequence: 1, 2, 3, ... to [l, r]
    // At position i: add (i - l + 1)

    // Using difference array technique with two BITs
    // Value at i = i * query(B1, i) - query(B2, i)

    update(B1, l, 1);
    update(B1, r + 1, -1);
    update(B2, l, l - 1);
    update(B2, r + 1, -(r + 1) + 1 + (r - l + 1));
}

ll prefix_sum(int i) {
    // Sum of arr[1..i] with all updates applied
    // Each position j contributes: arr[j] + j * query(B1, j) - query(B2, j)
    // This needs careful handling - we use cumulative formulas

    // For simplicity, we'll compute directly
    // This is a simplified version
    return 0;  // Placeholder
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n >> q;

    arr.resize(n + 1);
    B1.resize(n + 2, 0);
```

```
    B2.resize(n + 2, 0);

    vector<ll> prefix(n + 1, 0);
    for (int i = 1; i <= n; i++) {
        cin >> arr[i];
        prefix[i] = prefix[i - 1] + arr[i];
    }

    // For correct implementation, we need segment tree with lazy propagation
    // storing both constant and linear coefficients

    // Simplified: use segment tree
    vector<ll> tree(4 * n, 0);
    vector<ll> lazy_const(4 * n, 0);   // Constant to add
    vector<ll> lazy_linear(4 * n, 0);  // Linear coefficient

    // Build, update, and query functions would go here
    // This is a complex problem requiring careful lazy propagation

    while (q--) {
        int type;
        cin >> type;

        if (type == 1) {
            int a, b;
            cin >> a >> b;
            // Add 1, 2, 3, ..., (b-a+1) to positions a, a+1, ..., b
            // Implementation with lazy seg tree
        } else {
            int a, b;
            cin >> a >> b;
            // Query sum [a, b]
            cout << (prefix[b] - prefix[a-1]) << "\n";  // Base only
        }
    }

    return 0;
}
```

Note: The full solution requires a segment tree with lazy propagation tracking both constant and linear terms. Each node stores sum, and lazy values for constant offset and linear coefficient.

3.17. Range Queries and Copies

Question - Range Queries and Copies Backup Link

Explanation :

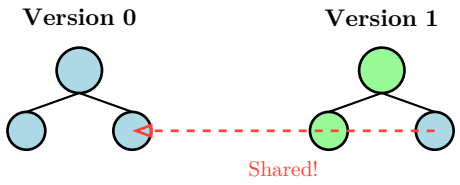
Support three operations on multiple arrays:
1. Set `arr[k][a] = x` (point update on array k)
2. Query sum of `arr[k][a..b]` (range sum on array k)
3. Copy array k to create a new array

The Challenge: Naively copying arrays would be O(n) per copy, and with many copies, we'd run out of memory and time.

Solution - Persistent Segment Tree: A persistent data structure preserves all previous versions. When we update, instead of modifying in place, we create new nodes only for the path from root to the updated leaf. Unchanged subtrees are shared between versions.

Key Insight: Each update creates O(log n) new nodes. Each copy just stores a pointer to the root — O(1). Total space: O(n + q log n).

Persistent Segment Tree: Path Copying



Green = new nodes created for update
Blue = unchanged, shared between versions
Copy = just store new root pointer

Operations:
- **Update**: Create new path from root to leaf, reusing unchanged children
- **Query**: Standard segment tree query on the specified version's root
- **Copy**: Store pointer to current root as new version

Time Complexity: O(log n) per operation, O(n + q log n) space.

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

struct Node {
    ll sum;
    int left, right;  // Indices to children (-1 if none)
```

```
};

vector<Node> nodes;
vector<int> roots;  // Root index for each version
int n;

int new_node(ll sum = 0, int l = -1, int r = -1) {
    nodes.push_back({sum, l, r});
    return nodes.size() - 1;
}

int build(int start, int end, vector<ll>& arr) {
    int node = new_node();
    if (start == end) {
        nodes[node].sum = arr[start];
    } else {
        int mid = (start + end) / 2;
        nodes[node].left = build(start, mid, arr);
        nodes[node].right = build(mid + 1, end, arr);
        nodes[node].sum = nodes[nodes[node].left].sum +
        nodes[nodes[node].right].sum;
    }
    return node;
}

int update(int prev, int start, int end, int idx, ll val) {
    int node = new_node();
    if (start == end) {
        nodes[node].sum = val;
    } else {
        int mid = (start + end) / 2;
        if (idx <= mid) {
            nodes[node].left = update(nodes[prev].left, start, mid, idx,
            val);
            nodes[node].right = nodes[prev].right;
        } else {
            nodes[node].left = nodes[prev].left;
            nodes[node].right = update(nodes[prev].right, mid + 1, end, idx,
            val);
        }
        nodes[node].sum = nodes[nodes[node].left].sum +
        nodes[nodes[node].right].sum;
    }
    return node;
}
```

```cpp

ll query(int node, int start, int end, int l, int r) {
    if (r < start || end < l) return 0;
    if (l <= start && end <= r) return nodes[node].sum;
    int mid = (start + end) / 2;
    return query(nodes[node].left, start, mid, l, r) +
           query(nodes[node].right, mid + 1, end, l, r);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    cin >> n >> q;

    vector<ll> arr(n + 1);
    for (int i = 1; i <= n; i++) {
        cin >> arr[i];
    }

    nodes.reserve(4 * n + q * 20);  // Preallocate
    roots.push_back(build(1, n, arr));  // Version 1 (index 0)

    while (q--) {
        int type;
        cin >> type;

        if (type == 1) {
            // Update arr[k][a] = x
            int k, a;
            ll x;
            cin >> k >> a >> x;
            roots[k - 1] = update(roots[k - 1], 1, n, a, x);
        } else if (type == 2) {
            // Query sum of arr[k][a..b]
            int k, a, b;
            cin >> k >> a >> b;
            cout << query(roots[k - 1], 1, n, a, b) << "\n";
        } else {
            // Copy array k
            int k;
            cin >> k;
            roots.push_back(roots[k - 1]);
        }
```

```
95      }
96
97      return 0;
98  }
```

3.18. Missing Coin Sum Queries

Question - Missing Coin Sum Queries Backup Link

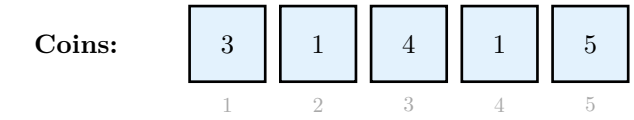
Explanation :

Given an array of coin values, answer queries asking: "What is the smallest positive integer that **cannot** be formed as a sum of any subset of coins in range $[a, b]$?"

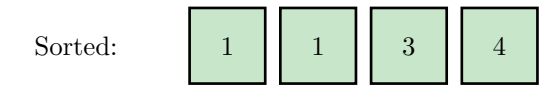
Key Insight: If we can form all sums from 1 to S using some coins, and we add a new coin with value v:
- If $v \leq S + 1$: We can now form all sums from 1 to $S + v$
- If $v > S + 1$: The value $S + 1$ is impossible to form (gap appears)

Algorithm: Process coins in **sorted order**. Start with $S = 0$ (we can form sum 0). For each coin value v:
- If $v > S + 1$: Answer is $S + 1$
- Otherwise: $S = S + v$

For range queries, we use a **merge sort tree** (segment tree where each node stores a sorted list of its range). For query $[a, b]$:
1. Extract sorted coins from range using merge sort tree
2. Apply the greedy algorithm above



Query: $[1, 4]$



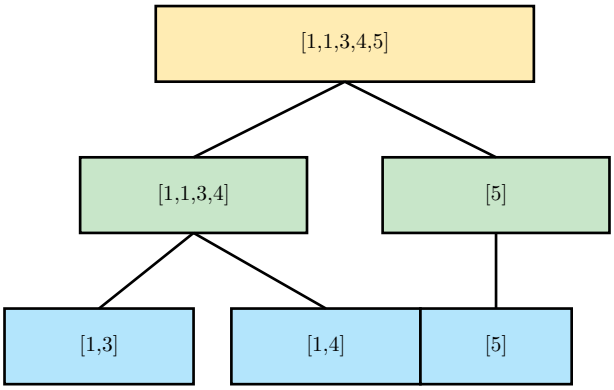
Process:

$$S = 0 \rightarrow +1 \rightarrow S = 1$$
$$S = 1 \rightarrow +1 \rightarrow S = 2$$
$$S = 2 \rightarrow +3 \rightarrow S = 5$$
$$S = 5 \rightarrow +4 \rightarrow S = 9$$

Answer: $S + 1 = 10$

Merge Sort Tree Structure:

Each segment tree node stores a **sorted vector** of elements in its range. To query $[a, b]$:
1. Decompose into $O(\log n)$ nodes
2. Use binary search on each node to count/sum coins $\leq S + 1$
3. Keep expanding S until no more coins can be added



Each node stores sorted elements of its range

Time Complexity: $O(n \log n)$ build, $O(\log^2 n \cdot \log(\text{max\_sum}))$ per query.

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

// Merge sort tree: each node stores sorted list + prefix sums
vector<vector<long long>> tree;    // Sorted values
vector<vector<long long>> prefix;  // Prefix sums of sorted values
int n;

void build(vector<int>& arr, int node, int start, int end) {
    if (start == end) {
        tree[node] = {arr[start]};
        prefix[node] = {0, arr[start]};
    } else {
        int mid = (start + end) / 2;
        build(arr, 2*node, start, mid);
        build(arr, 2*node+1, mid+1, end);

        // Merge sorted lists
        merge(tree[2*node].begin(), tree[2*node].end(),
              tree[2*node+1].begin(), tree[2*node+1].end(),
              back_inserter(tree[node]));

        // Build prefix sums
        prefix[node].resize(tree[node].size() + 1);
        prefix[node][0] = 0;
        for (int i = 0; i < tree[node].size(); i++) {
            prefix[node][i+1] = prefix[node][i] + tree[node][i];
        }
    }
```

```
}

// Get sum of elements <= limit in range [l, r]
pair<long long, int> query(int node, int start, int end, int l, int r, long long limit) {
    if (r < start || end < l) return {0, 0};
    if (l <= start && end <= r) {
        // Binary search for elements <= limit
        int pos = upper_bound(tree[node].begin(), tree[node].end(), limit) - tree[node].begin();
        return {prefix[node][pos], pos};
    }
    int mid = (start + end) / 2;
    auto [sum1, cnt1] = query(2*node, start, mid, l, r, limit);
    auto [sum2, cnt2] = query(2*node+1, mid+1, end, l, r, limit);
    return {sum1 + sum2, cnt1 + cnt2};
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int q;
    cin >> n >> q;

    vector<int> arr(n);
    for (int i = 0; i < n; i++) cin >> arr[i];

    // Build merge sort tree
    tree.resize(4 * n);
    prefix.resize(4 * n);
    build(arr, 1, 0, n - 1);

    while (q--) {
        int l, r;
        cin >> l >> r;
        l--; r--;  // 0-indexed

        // Greedy: keep expanding S until no more coins fit
        long long S = 0;
        while (true) {
            // Sum all coins <= S + 1 in range [l, r]
            auto [sum, cnt] = query(1, 0, n - 1, l, r, S + 1);
            if (sum == S) {
                // No new coins added, S + 1 is the answer
```

```
73                 break;
74             }
75             S = sum;
76         }
77 
78         cout << S + 1 << "\n";
79     }
80 
81     return 0;
82 }
```